

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)» (МАИ)

Направление подготовки: 27.03.05 «Инноватика»

Лабораторная работа №1

по дисциплине «Структура программного обеспечения и базы данных»

Выполнили:

Студенты гр. МЗО-236БВ-24

Хаитова К.Н

Чулкова В.А

Преподаватель:

Александрова С.С.

Прудников А.В.

Москва 2025

Оглавление

Цель работы:	3
ССЫЛКА НА ГИТХАБ.....	11
Как работает:	11
Вывод:	15

Цель работы:

Приложение на PyQT для визуализации данных

Загрузка датасета через интерфейс

1 вкладка - статистика по данным загруженным в бд, возможность выбрать таблицу

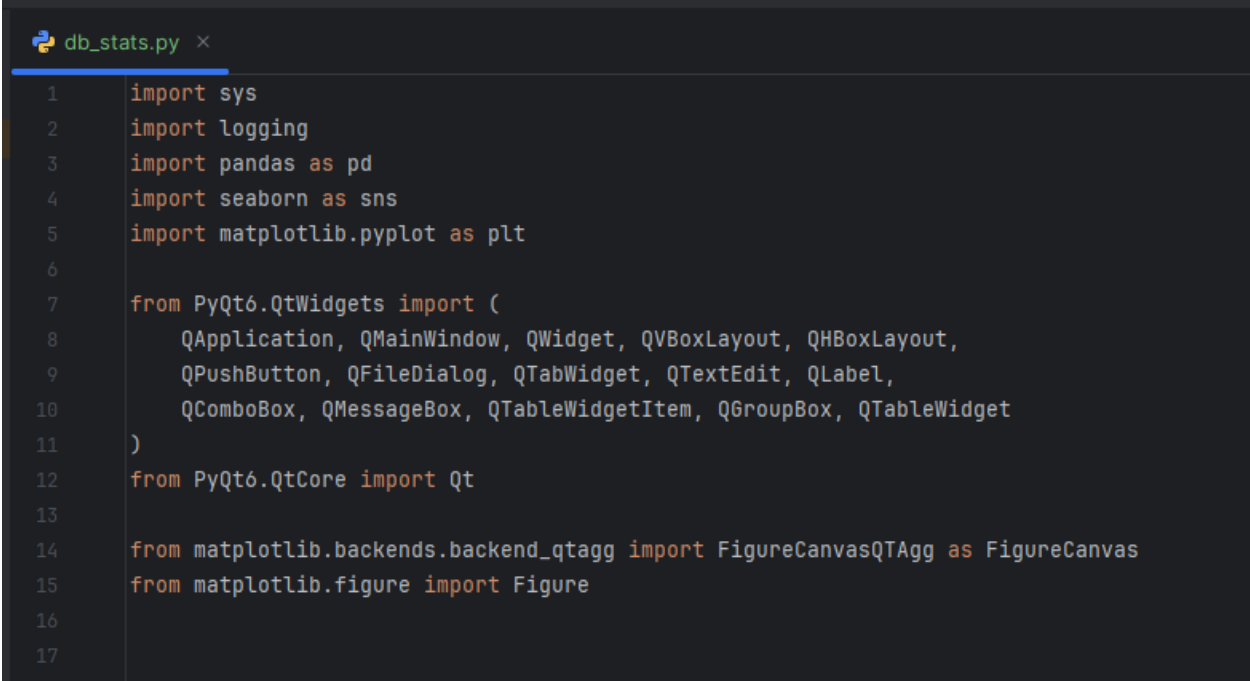
2 вкладка - графики корреляции числовых параметров (сиборн)

3 вкладка - тепловая карта

4 вкладка - выбираем любой числовой столбец, строим по нему линейный график

5 вкладка - ход работы(по сути лог действий пользователя)

1. Импортируем нужные библиотеки



```
db_stats.py ×
1 import sys
2 import logging
3 import pandas as pd
4 import seaborn as sns
5 import matplotlib.pyplot as plt
6
7 from PyQt6.QtWidgets import (
8     QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
9     QPushButton, QFileDialog, QTabWidget, QTextEdit, QLabel,
10     QComboBox, QMessageBox, QTableWidgetItem, QGroupBox, QTableWidget
11 )
12 from PyQt6.QtCore import Qt
13
14 from matplotlib.backends.backend_qtagg import FigureCanvasQTagg as FigureCanvas
15 from matplotlib.figure import Figure
16
17
```

2. Часть для настройки логирования и создания пользовательского графического холста Matplotlib для встраивания

```
0 #логирование
1 logging.basicConfig(
2     level=logging.INFO,
3     format="%(asctime)s - %(levelname)s - %(message)s"
4 )
5 logger = logging.getLogger(__name__)
6
7
8
9 #matplotlib графики
10 class MplCanvas(FigureCanvas): 2 usages new *
11     def __init__(self, width=5, height=4, dpi=100): new *
12         self.fig = Figure(figsize=(width, height), dpi=dpi)
13         self.ax = self.fig.add_subplot(111)
14         super().__init__(self.fig)
```

3. Главное окно

```
37
38 #главное окно
39 class DataVizApp(QMainWindow): 1 usage new *
40     def __init__(self): new *
41         super().__init__()
42         self.setWindowTitle("Дата-визуализатор")
43         self.resize(1200, 800)
44
45         self.df: pd.DataFrame | None = None
46         self.corr_plotted = False
47
48         main_widget = QWidget()
49         self.setCentralWidget(main_widget)
50         main_layout = QVBoxLayout(main_widget)
51
52         load_btn = QPushButton("Загрузить датасет (CSV)")
53         load_btn.clicked.connect(self.load_dataset)
54         main_layout.addWidget(load_btn)
55
56         self.tabs = QTabWidget()
57         self.tabs.currentChanged.connect(self.on_tab_changed)
58         main_layout.addWidget(self.tabs)
59
60         self.init_tab_statistics()
61         self.init_tab_correlation()
62         self.init_tab_heatmap()
63         self.init_tab_lineplot()
64         self.init_tab_logs()
65
66
```

4. Вкладка статистика

```
67
68 #статистика
69 def init_tab_statistics(self): 1 usage new *
70     self.tab_stats = QWidget()
71     layout = QVBoxLayout(self.tab_stats)
72
73     self.num_group = QGroupBox("Числовые признаки")
74     self.cat_group = QGroupBox("Категориальные признаки")
75
76     self.num_table = QTableWidgetItem()
77     self.cat_table = QTableWidgetItem()
78
79     num_layout = QVBoxLayout()
80     num_layout.addWidget(self.num_table)
81     self.num_group.setLayout(num_layout)
82
83     cat_layout = QVBoxLayout()
84     cat_layout.addWidget(self.cat_table)
85     self.cat_group.setLayout(cat_layout)
86
87     layout.addWidget(self.num_group)
88     layout.addWidget(self.cat_group)
89
90     self.tabs.addTab(self.tab_stats, "Статистика")
```

Организация таблиц

```
91
92 def fill_table(self, table: QTableWidgetItem, df: pd.DataFrame): 2 usages new *
93     table.clear()
94
95     table.setRowCount(df.shape[0])
96     table.setColumnCount(df.shape[1] + 1)
97
98     table.setHorizontalHeaderLabels(["Метрика"] + list(df.columns))
99
100     for row_idx, row_name in enumerate(df.index):
101         table.setItem(row_idx, column: 0, QTableWidgetItem(str(row_name)))
102
103         for col_idx, value in enumerate(df.loc[row_name]):
104             table.setItem(
105                 row_idx,
106                 col_idx + 1,
107                 QTableWidgetItem(str(round(value, 3)) if isinstance(value, float) else str(value))
108             )
109
110     table.resizeColumnsToContents()
```

5. Вкладка корреляции

```
111
112     #корреляции
113     def init_tab_correlation(self): 1 usage new *
114         self.tab_corr = QWidget()
115         layout = QVBoxLayout(self.tab_corr)
116
117         label = QLabel("корреляционные графики")
118         label.setAlignment(Qt.AlignmentFlag.AlignCenter)
119         layout.addWidget(label)
120
121         self.tabs.addTab(self.tab_corr, "Корреляции")
```

6. Вкладка тепловая карта

```
123
124     #тепловая карта
125     def init_tab_heatmap(self): 1 usage new *
126         self.tab_heat = QWidget()
127         layout = QVBoxLayout(self.tab_heat)
128
129         self.heat_canvas = MplCanvas()
130         layout.addWidget(self.heat_canvas)
131
132         self.tabs.addTab(self.tab_heat, "Тепловая карта")
133
```

7. Вкладка линейные графики

```
135
136     #линейные графики
137     def init_tab_lineplot(self): 1 usage new *
138         self.tab_line = QWidget()
139         layout = QVBoxLayout(self.tab_line)
140
141         top = QHBoxLayout()
142         top.addWidget(QLabel("Числовой столбец:"))
143
144         self.column_selector = QComboBox()
145         top.addWidget(self.column_selector)
146
147         btn = QPushButton("Построить график")
148         btn.clicked.connect(self.plot_line)
149         top.addWidget(btn)
150
151         layout.addLayout(top)
152
153         self.line_canvas = MplCanvas()
154         layout.addWidget(self.line_canvas)
155
156         self.tabs.addTab(self.tab_line, "Линейный график")
157
```

8. Вкладка логов

```
160     #логи
161     def init_tab_logs(self): 1 usage new *
162         self.tab_logs = QWidget()
163         layout = QVBoxLayout(self.tab_logs)
164
165         self.log_view = QTextEdit()
166         self.log_view.setReadOnly(True)
167         layout.addWidget(self.log_view)
168
169         self.tabs.addTab(self.tab_logs, "Ход работы")
170
171     #лог
172     def log(self, message: str): 5 usages new *
173         logger.info(message)
174         self.log_view.append(message)
```

9. Загрузка датасета

```
177
178     #загрузка датасета
179     def load_dataset(self): 1 usage new *
180         path, _ = QFileDialog.getOpenFileName(
181             self, "Выберите CSV файл")
182         if not path:
183             return
184
185         try:
186             self.df = pd.read_csv(path)
187             self.corr_plotted = False
188             self.log(f"Загружен датасет: {path}")
189             self.update_statistics()
190             self.update_column_selector()
191         except Exception as e:
192             QMessageBox.critical(self, "Ошибка", str(e))
193
```

10. Обновление статистики

```
194
195     #обновление статистики
196     def update_statistics(self): 1 usage new *
197         if self.df is None:
198             return
199
200         num_df = self.df.select_dtypes(include="number")
201         cat_df = self.df.select_dtypes(exclude="number")
202
203         if not num_df.empty:
204             num_desc = num_df.describe()
205             self.fill_table(self.num_table, num_desc)
206
207         if not cat_df.empty:
208             cat_desc = cat_df.describe()
209             self.fill_table(self.cat_table, cat_desc)
210
211         self.log("Обновлена статистика (табличный вид)")
212
```


11. Переключение вкладок

```
212
213     # переключение
214     def on_tab_changed(self, index: int): 1 usage new *
215         if self.df is None:
216             return
217
218         widget = self.tabs.widget(index)
219
220         if widget == self.tab_corr and not self.corr_plotted:
221             self.plot_seaborn_pairplot()
222
223         elif widget == self.tab_heat:
224             self.plot_heatmap()
225
```

12. Обновление корреляционных графиков

```
226
227     #корреляционные графики
228     def plot_seaborn_pairplot(self): 1 usage new *
229         num_df = self.df.select_dtypes(include="number")
230
231         if num_df.shape[1] < 2:
232             QMessageBox.warning(
233                 self,
234                 "Недостаточно данных",
235                 "Для корреляции нужно минимум 2 числовых столбца"
236             )
237             return
238
239         sns.pairplot(num_df)
240         plt.show()
241
242         self.corr_plotted = True
243         self.log("Открыто окно корреляций")
```

13. Обновление тепловой карты

```
245
246 #тепловая карта
247 def plot_heatmap(self): 1 usage new *
248     self.heat_canvas.ax.clear()
249
250     corr = self.df.select_dtypes(include="number").corr()
251     sns.heatmap(
252         corr,
253         ax=self.heat_canvas.ax,
254         annot=True,
255         cmap="coolwarm",
256         fmt=".2f"
257     )
258
259     self.heat_canvas.draw()
260     self.log("Построена тепловая карта")
261
```

14. Обновление линейных графиков

```
262
263 #линейные графики
264 def update_column_selector(self): 1 usage new *
265     self.column_selector.clear()
266     self.column_selector.addItem(
267         self.df.select_dtypes(include="number").columns
268     )
269
270 def plot_line(self): 1 usage new *
271     col = self.column_selector.currentText()
272     self.line_canvas.ax.clear()
273     self.line_canvas.ax.plot(self.df[col])
274     self.line_canvas.ax.set_title(col)
275     self.line_canvas.draw()
276     self.log(f"Построен линейный график: {col}")
277
```

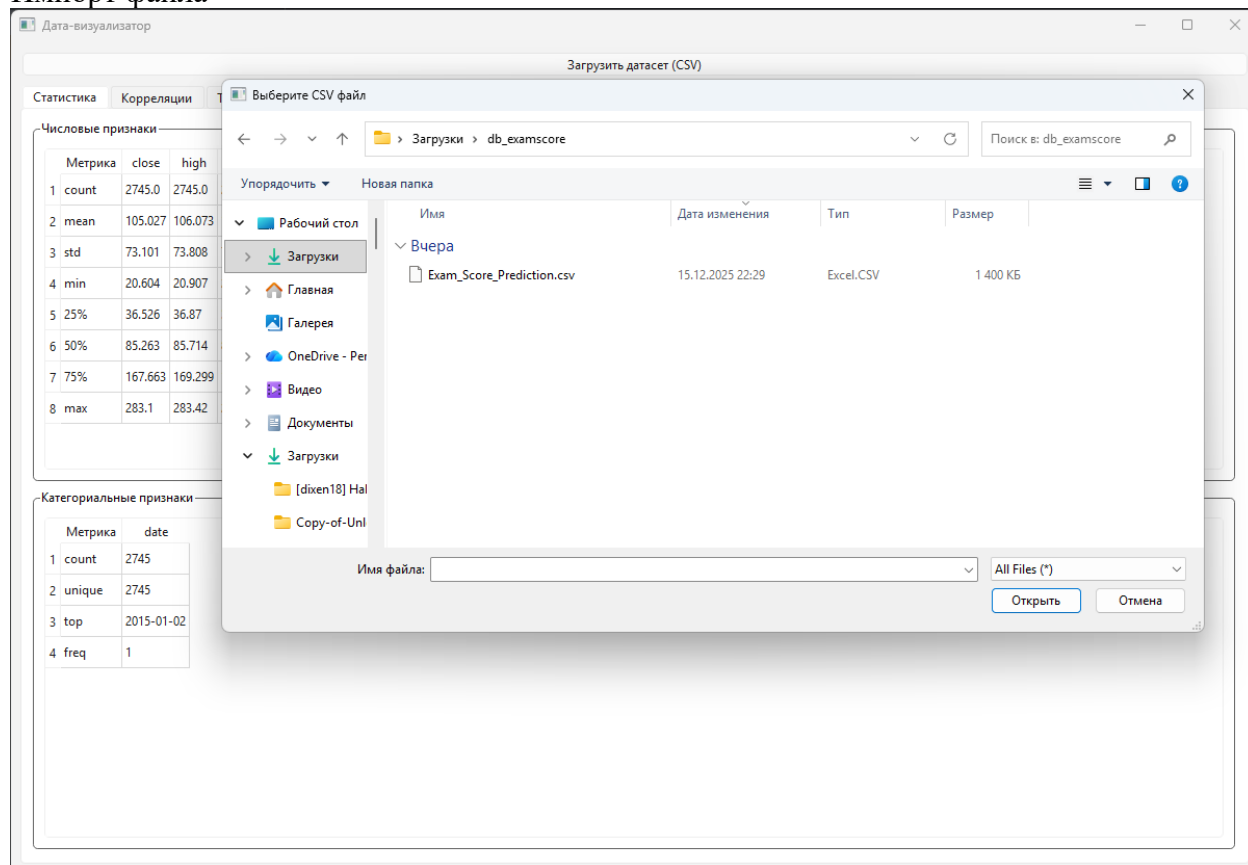
15. Запуск

```
279 # запуск
280 ▶ if __name__ == "__main__":
281     app = QApplication(sys.argv)
282     window = DataVizApp()
283     window.show()
284     sys.exit(app.exec())
285
```

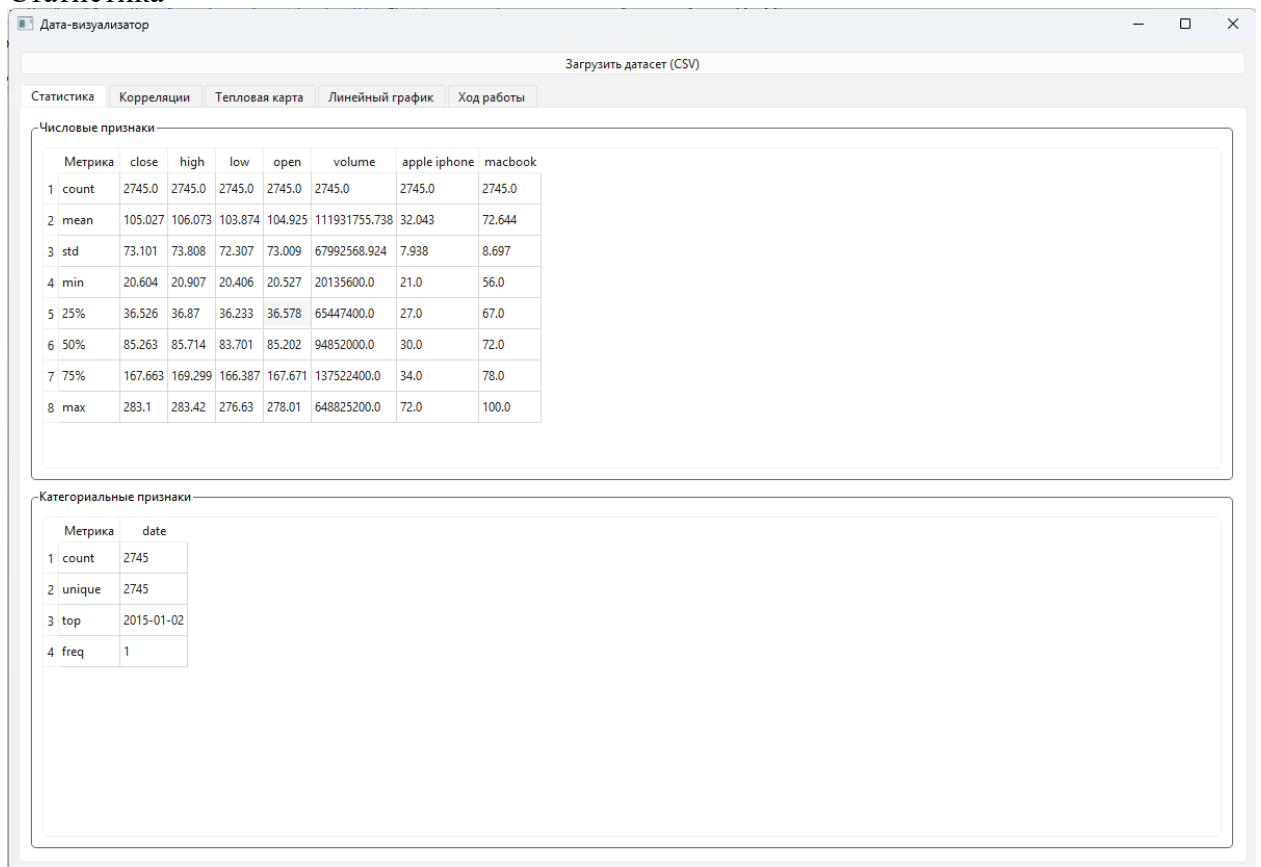
ССЫЛКА НА ГИТХАБ https://github.com/spobd25course/LR1_db.git

Как работает:

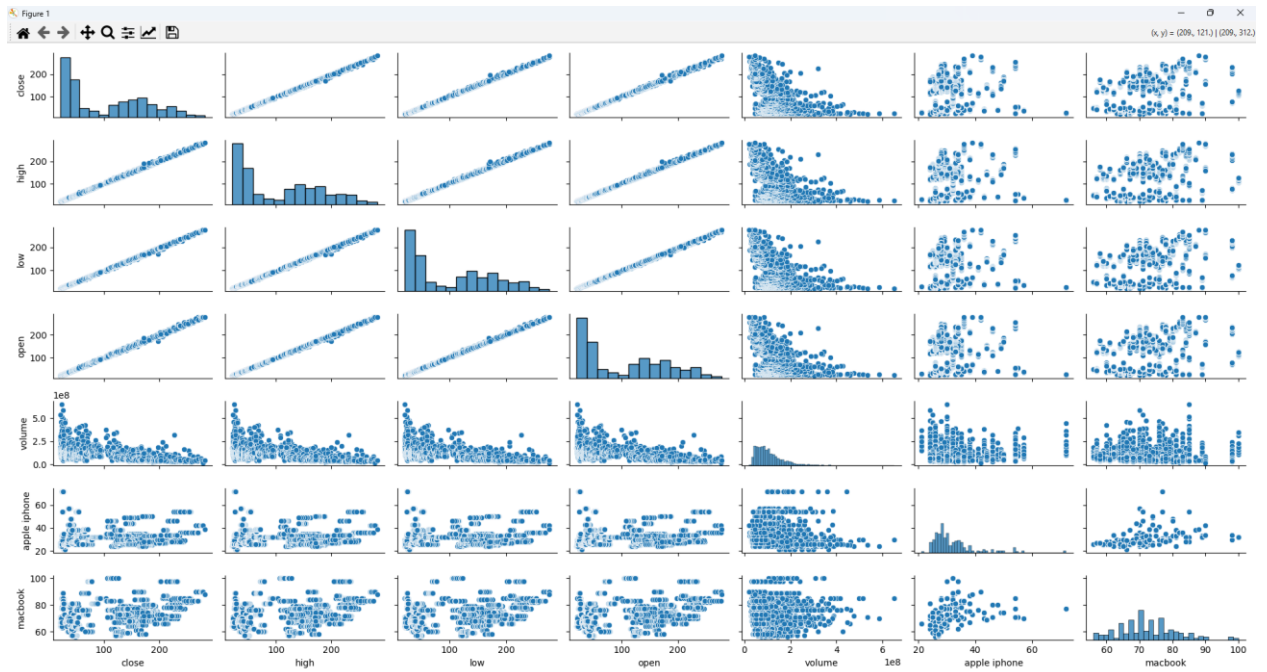
Импорт файла

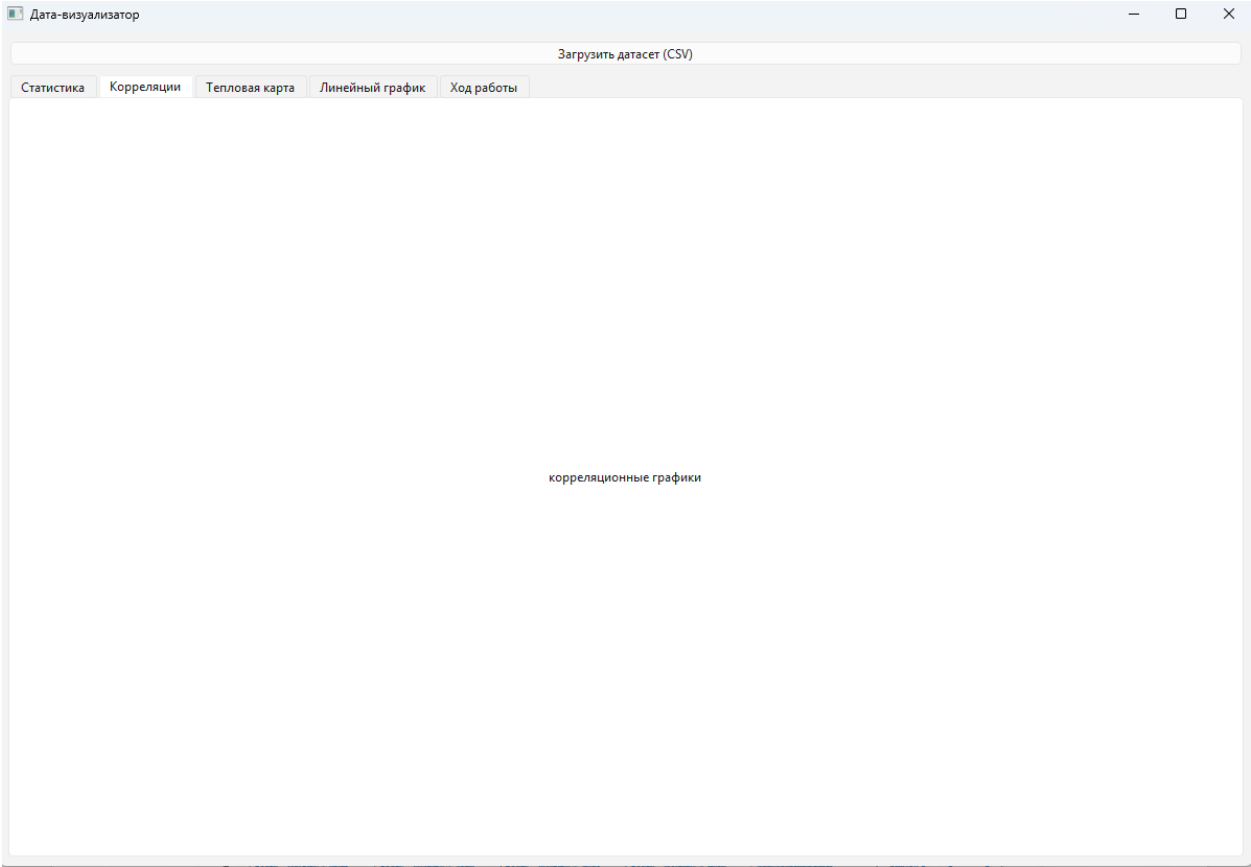


Статистика

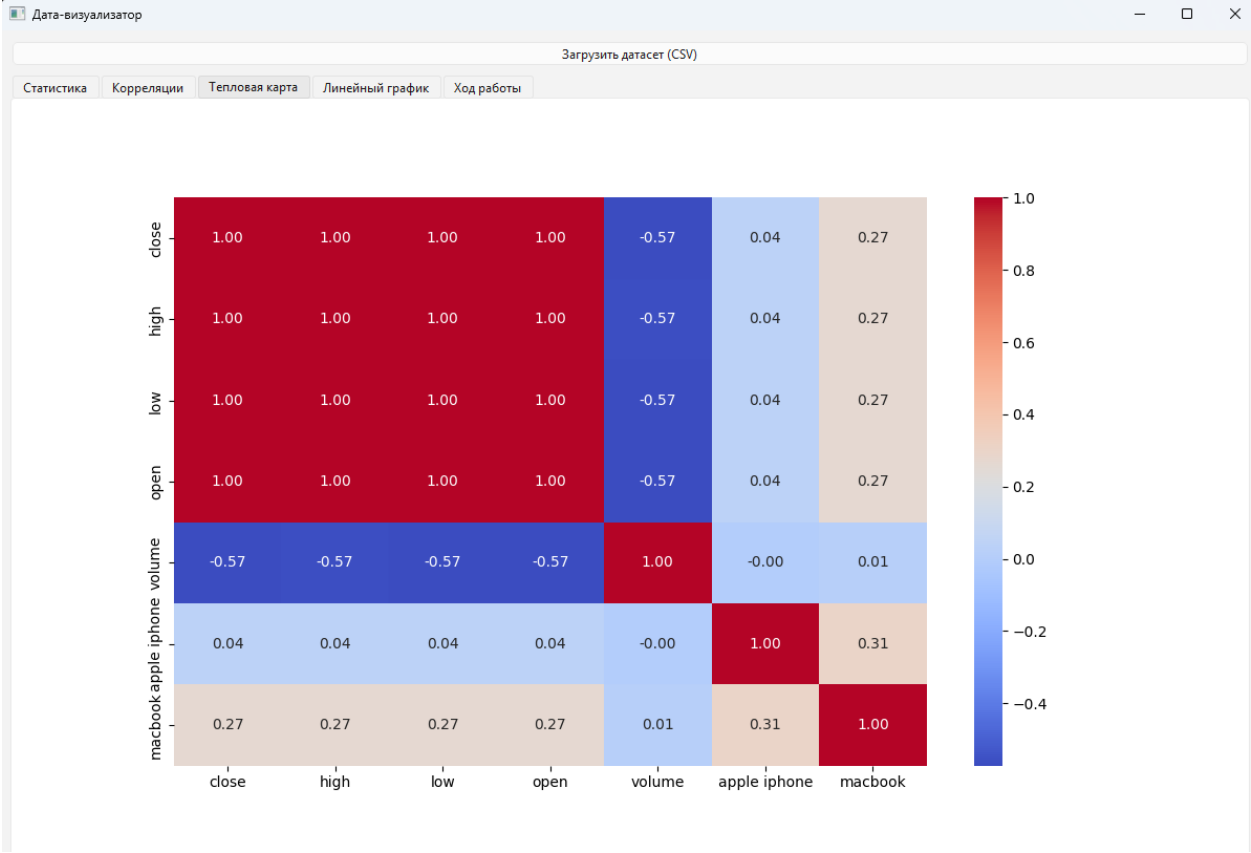


Корреляции (можно реализовать только в отдельном matplotlib-окне)

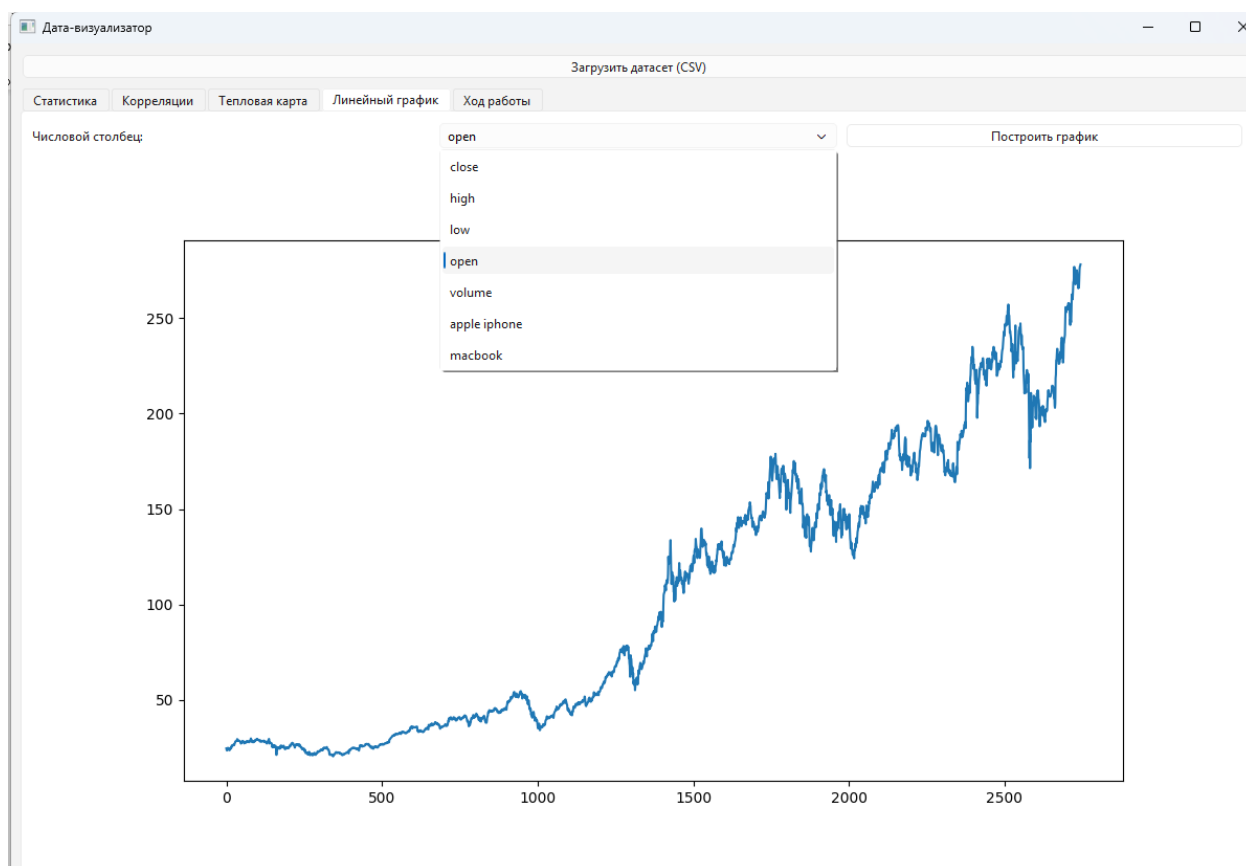




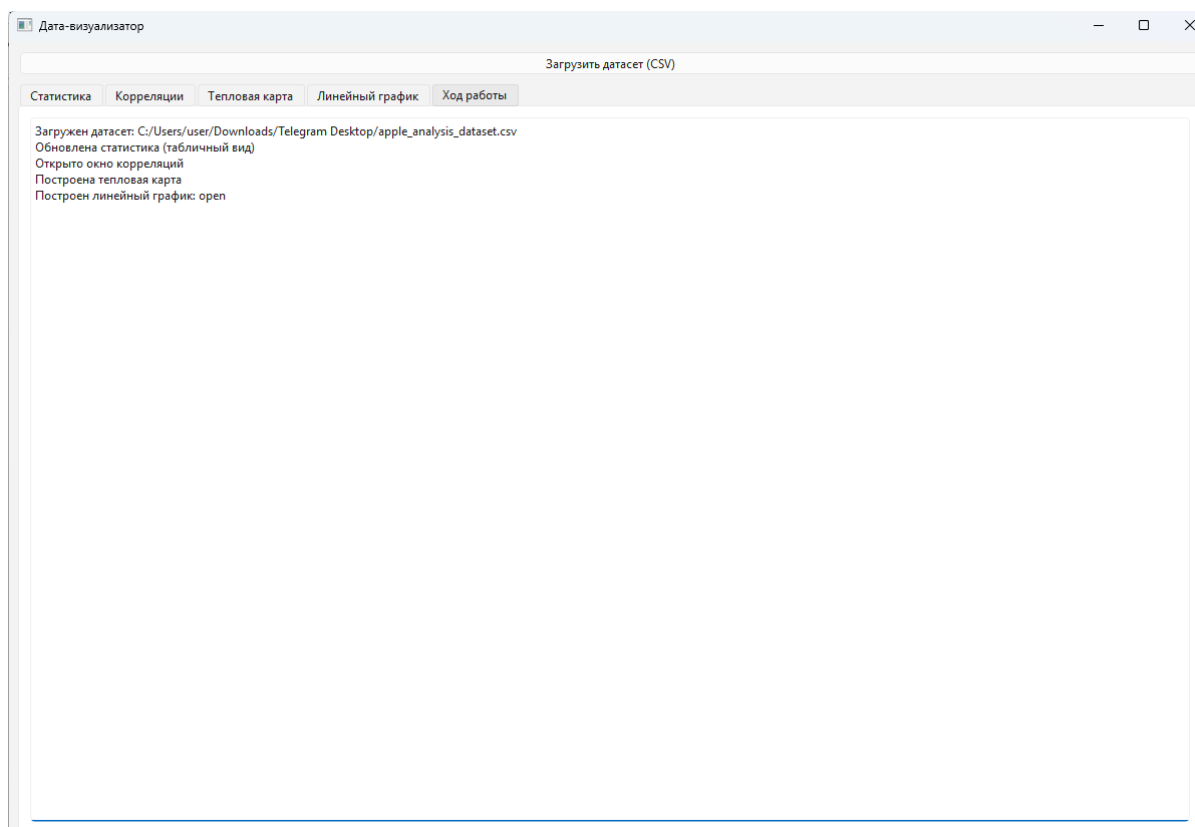
Тепловая карта



Линейный график (можем выбрать нужное списком)



Логи



Вывод:

В ходе лабораторной работы было разработано приложение с использованием библиотеки PyQt6, предназначенное для анализа и визуализации табличных данных. Приложение позволяет загружать датасеты в формате CSV и автоматически выполнять их первичное исследование. В процессе работы реализован вывод описательной статистики для числовых и категориальных признаков в наглядном табличном виде. Для анализа взаимосвязей между числовыми параметрами используется библиотека Seaborn, обеспечивающая построение корреляционных графиков в отдельном окне. Также в приложении реализована визуализация корреляционной матрицы в виде тепловой карты. Пользователь имеет возможность строить линейные графики по выбранным числовым признакам. Для повышения удобства работы была добавлена система логирования, отображающая ход выполнения операций. Разработанное приложение может быть использовано как инструмент для первичного анализа данных и демонстрирует практическое применение методов визуальной аналитики.